

Sleep VS GPA in College: Predicción del GPA en base al sueño en estudiantes de carrera.

Alfredo Alejandro Soto Herrera - A01711368

Abstract – En el presente documento se implementa un modelo de Machine Learning con el fin de lograr predecir el GPA de estudiantes universitarios en base a las horas de sueño que diariamente tienen, además de su calidad, etc. Esto con el fin de encontrar una correlación entre ambas situaciones, más no una causalidad. Se utilizó un modelo de regresión lineal multivariable junto con una función de activación ReLU, MSE para calcular el costo o pérdida y Batch Back Propagation para corregir todos los pesos y sesgos. Además de este modelo, se utiliza Random Forest para medir las diferencias que tiene con la regresión lineal, comparando en qué algoritmo se ajusta mejor nuestro modelo y datos.

Keywords – IA, Machine learning, Dataset, ETL, EDA, Regresión lineal, ReLU, MSE, Bias, Random Forest.

I. INTRODUCCIÓN

La inteligencia artificial es un concepto que ha rondado en la humanidad desde hace muchos años, se cree que es algo reciente pero sus inicios se pueden remontar desde el siglo XVIII, esto debido a que en 1726 la novela titulada como “Los viajes de Gulliver”, escrita por Jonathan Swift habla de un concepto nuevo en donde se introduce a una “máquina” que ayuda a los eruditos a escribir y a tener nuevas ideas [1]. Desde entonces podemos estar hablando de algo no humano lo cual ayuda a la sociedad.

A pesar de tener ideas de IA, el concepto no se aterrizó hasta que en 1950 Alan Turing publica “Computing Machinery and Intelligence” en donde se dejan estas ideas fugaces que pueden interpretarse hoy en día como conceptos relacionados a IA, sino que todo se concentra en este concepto. Turing plantea si en algún punto las máquinas podrán exhibir un comportamiento inteligente sin el apoyo humano [1]. Además de crear la prueba de Turing para poder medir la inteligencia.

Turing sentó las bases e hizo que muchos científicos se pusieran a investigar en esta área, ya que le veían potencial, pero durante casi 30 años de investigación, no se logró a los resultados esperados, se creó el perceptrón, lo que conocemos como una neurona en Deep learning. Pero no fue hasta que IBM logró vencer al campeón de ajedrez en 1997 [2]. Esto despertó al mundo después de esos 30 años de avances sin resultados, entonces en 2012 AlexNet vuelve a alborotar el

avispero [3], ganó la competencia ImageNet con un amplio margen con respecto al segundo lugar y así sucedían cosas que convencían a la comunidad científica que la Inteligencia Artificial tenía un camino claro, Machine Learning.

Machine Learning no es otra cosa que un área dentro de la Inteligencia Artificial, con los casos de éxito anteriores, es que se ha seguido el camino de ML (abreviatura de Machine Learning) a pesar de existir otras áreas. Lo que hace ML a grandes rasgos es que puede aprovechar un gran número de datos y aplicar con ellos una serie de algoritmos para poder generar un modelo que pueda detectar patrones en los datos y de esta forma poder predecir nuevas relaciones en datos desconocidos [4].

El objetivo de este paper es trabajar dentro del área de Machine Learning para poder crear un modelo y hacer predicciones con él dentro de un dataset. El dataset con el que se estará trabajando fue extraído de Kaggle [5], titulado como “Sleep vs. GPA in College”, en este dataset, se analiza el promedio de estudiantes universitarios contrastándolo con sus horas y calidad de sueño.

Analizar este caso es fundamental ya que la presión que sufren los estudiantes por obtener buenas calificaciones puede llegar a derivar en problemas de depresión o incluso de ideación suicida, en la UNAM se mostró que cerca del 39.6% de los estudiantes tienen ideas suicidas [6]. Está claro que no es la única causa o la causa principal de esto, pero mientras se puedan reducir las posibilidades, este trabajo tiene un valor.

Se utilizará un modelo de regresión lineal para poder entrenar un modelo que nos permita predecir el promedio estudiantil en base a su sueño.

II. EDA Y ETL

A. EDA inicial

EDA, significa “Análisis Exploratorio de los Datos”, nos permite conocer con lo que estamos trabajando y poder encontrar formas de interpretar los datos de manera visual o numérica. En este caso se hará un EDA inicial para conocer los datos y así poder realizar un ETL con certeza de lo que se está haciendo en él, finalmente se realiza un segundo EDA

para corroborar que los datos se encuentren limpios y cómo los deseamos.

El dataset elegido [6], cuenta con 21 features y 1 variable a predecir y 634 instancias.

Nombre	Descripción
student_id	ID anónimo por estudiante, es único.
study	Identificador de universidad.
university	Universidad a la que el estudiante atiende.
semester	Número de semestre en el que se hizo la medición.
cohort_code	Abreviaturas para identificar a las universidades.
gender	Género reportado por la misma universidad.
first_generation	Indica 1 si ninguno de sus padres completó o inició la universidad y 0 si alguno de los 2 si la terminó.
underrepresented	Indica 1 si alguno de sus padres pertenece a alguna minoría (negro, latino, hispano, nativo americano, de alguna isla del pacífico). 0 si no aplica.
avg_sleep_minutes	Promedio en minutos de lo que duerme el estudiante, sin contar el tiempo recostado antes de dormir.
avg_sleep_hours	Promedio del sueño, pero en este caso en horas y no minutos.
daytime_sleep_minutes	Es el tiempo que se duerme en el día medido en minutos.
sleep_midpoint_minutes	La hora promedio que queda a la mitad del periodo de sueño, expresada en minutos desde medianoche.
bedtime_variability	Qué tanto varía la hora a la que el estudiante se va a dormir entre diferentes noches.
nights_tracked_fraction	Proporción de noches en las que Fitbit registró datos de sueño válidos durante el periodo de estudio.
prior_gpa	GPA promedio del periodo anterior.
term_gpa	GPA obtenido por el estudiante en este periodo.
gpa_change	Resta entre el prior_gpa y term_gpa para ver la razón de cambio entre ellos.
term_units	Cantidad de créditos académicos cursados en el periodo.
term_load_z	Indica qué tan por encima o por debajo del promedio está la carga académica.
under 6h sleep	Marca con 1 si el estudiante

	duerme menos de 6 horas al día, con 0 si tiene 6 o más horas.
sleep_bracket	Divide las horas que duerme el estudiante en rangos.
sleep_midpoint_clock	Lo mismo que sleep_midpoint_minutes, pero expresado como hora legible en lugar de minutos.

Tabla 1. Variables en el Dataset.

Conociendo las variables y su función nos podemos dar una idea de las columnas que vamos a eliminar porque no nos van a servir para predecir “term_gpa”, pero antes de ello vamos a analizar un poco más los datos.

En este EDA inicial pudimos revisar los features que contaban con valores nulos, si se repetían instancias, etc. Todo con el fin de poder descubrir como vienen los datos y que features no necesitamos.

Valores Faltantes	Valores faltantes	Porcentaje
gender	3	0.473186
first_generation	4	0.630915
underrepresented	1	0.157729
term_units	147	23.186120
term_load_z	147	23.186120

Figura 1. Features con valores faltantes.

En la Figura 1 podemos ver que hay 5 features con valores faltantes, pero en 3 de ellas no hay problema porque son pocos valores, en cambio en “term_units” y en “term_load_z” falta un 23% de los datos, lo cual si es una gran cantidad como para simplemente completar con un promedio. En el sitio de Kaggle en donde se encuentra el dataset [5], se nos avisa que faltaban esos datos porque pertenecen a una universidad que no los dio.

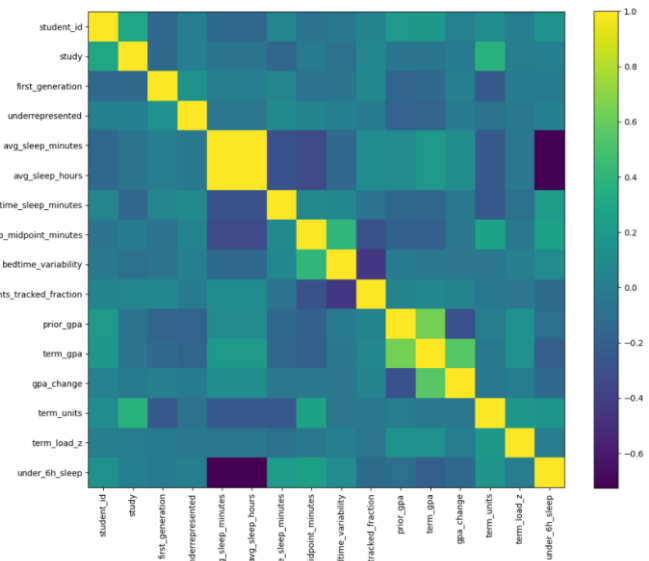


Figura 2. Matriz de Correlación

De igual forma se hizo una matriz de correlación para poder observar que features no sirven para poder predecir la variable “term_gpa”, en donde se pudo encontrar que no hay algún feature que tenga una relación clara con “term_gpa”, con lo cual dependemos de más de una variable para poderla predecir.

```
Correlación con GPA
term_gpa      1.000000
prior_gpa     0.638025
gpa_change    0.548917
avg_sleep_minutes 0.201674
avg_sleep_hours 0.201542
student_id    0.196675
term_load_z   0.140652
nights_tracked_fraction 0.074180
term_units    -0.030672
bedtime_variability -0.035967
study         -0.056451
first_generation -0.135061
daytime_sleep_minutes -0.153059
underrepresented -0.170013
sleep_midpoint_minutes -0.194537
under_6h_sleep -0.199581
Name: term_gpa, dtype: float64
```

Figura 3. Valores de Correlación con GPA.

Hay que aclarar que los features que más relación tienen son “prior_gpa” y “gpa_change” las cuales también muestran el GPA, con lo cual surge la posibilidad de eliminarlas ya que nos pueden generar data leakage.

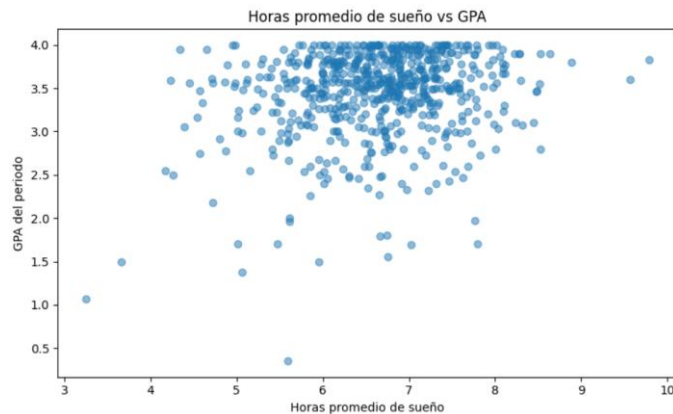


Figura 4. Scatterplot de las Horas de Sueño contra GPA.

Finalmente se realizó un Scatterplot para poder ver si llega a haber una relación entre las horas de sueño y el GPA, en donde podemos notar que la mayor parte de los datos se encuentra entre 6 y 7 horas de sueño de 3.0 GPA para arriba, podemos observar casos en donde a pesar de dormir 8 horas no logró un alto GPA, o con pocas horas de sueño si tiene un GPA muy alto.

B. ETL

Para comenzar con el proceso de transformación en el ETL, lo primero fue borrar aquellas features que nos dimos cuenta de que no necesitábamos.

Columna Eliminada	Razón
-------------------	-------

student_id	Es un identificador para los estudiantes, no nos ayuda a predecir.
study	Es otro identificador para las universidades.
university	No necesitamos el nombre de la universidad para predecir.
semester	Saber el semestre en el que está el estudiante tampoco ayuda para poder predecir.
cohort_code	Otro identificador de universidades que no se necesita.
avg_sleep_minutes	Es lo mismo que “avg_sleep_hours”, entonces es redundante.
gpa_change	Al derivar de “term_gpa” ya que es la resta de este con el anterior GPA, puede ocasionar data leakage.
term_units	Tiene 147 valores faltantes y se decidió no hacer imputación.
term_load_z	Igual que el caso anterior, tiene muchos datos faltantes.
under_6h_sleep	La información que nos da es redundante ya que la podemos obtener directamente de “avg_sleep_hours”.
sleep_bracket	Igual que el caso anterior, es redundante a “avg_sleep_hours”.
sleep_midpoint_clock	Es la representación de “sleep_midpoint_minutes” pero en formato de horario, por lo que nos quedamos con la variable numérica.

Tabla 2. Features Eliminadas

Como se observó en la Figura 1, teníamos 5 variables con valores faltantes, después de eliminar features nos quedamos solamente con 3, en los 3 casos realizamos una imputación utilizando la moda, ya que las 3 variables se tratan casos binarios, hombre o mujer en “género”, 1 o 0 en “underrepresented” y de igual forma 0 y 1 en “first_generation”, por lo que al usar la moda simplemente se agarra el valor con mayor frecuencia. En este último se detectó un caso inválido, ya que hubo una sola respuesta con valor de 2, lo cual no es válido, entonces se transformó a NA para hacer la imputación junto con los otros 4 valores NA.

	variable	Q1	Q3	IQR	limite_inferior	\
0	first_generation	0.00000	0.0000	0.00000	0.000000	
1	underrepresented	0.00000	0.0000	0.00000	0.000000	
2	avg_sleep_hours	6.11250	7.1700	1.05750	4.526250	
3	daytime_sleep_minutes	23.10000	51.2500	28.15000	-19.125000	
4	sleep_midpoint_minutes	345.17500	437.6250	92.45000	206.500000	
5	bedtime_variability	0.07470	0.2917	0.21700	-0.250800	
6	nights_tracked_fraction	0.82100	1.0000	0.17900	0.552500	
7	prior_gpa	3.23225	3.7895	0.55725	2.396375	
8	term_gpa	3.23300	3.8100	0.57700	2.367500	
	limite_superior	cantidad_outliers	porcentaje_outliers			
0	0.000000	103	16.246057			
1	0.000000	119	18.769716			
2	8.756250	11	1.735016			
3	93.475000	29	4.574132			
4	576.300000	15	2.365931			
5	0.617200	94	14.826498			
6	1.268500	54	8.517350			
7	4.625375	17	2.681388			
8	4.675500	21	3.312303			

Figura 5. Análisis en busca de Outliers.

Después de la imputación, hay que buscar outliers en los datos, por ello es por lo que calculando el primer y tercer cuartil es que logramos identificar el número de outliers como se observa en la Figura 5, ahora depende de nosotros ver que outliers reportados en verdad lo son y no solo valores atípicos pero posibles por la naturaleza de la variable. Por ejemplo, “avg_sleep_hours” tiene 11 outliers marcados, pero con un máximo de 9.7 horas, lo cual es algo más que posible y un inferior de 4.5 horas lo cual sí es poco, pero aun así es algo que pasa por lo cual no hay outliers en esta variable.

Las variables que sí son un poco sospechosas son “daytime_sleep_minutes”, “sleep_midpoint_minutes” y “bedtime_variability”. Con lo cual, rescatamos una serie de boxplots hechos en el proceso de EDA Inicial.

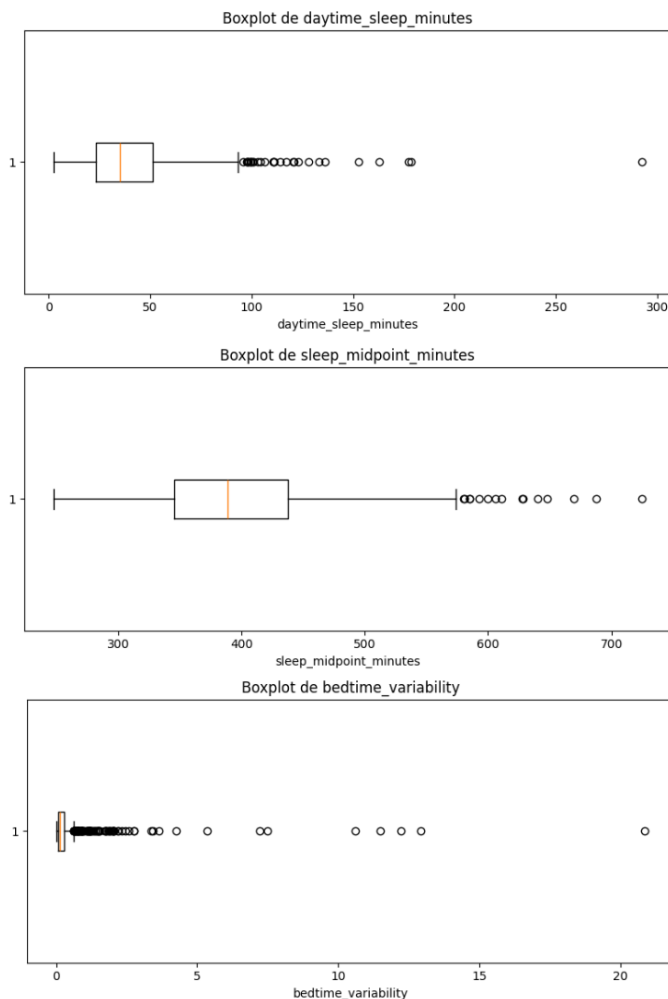


Figura 6. Box Plots de variables sospechosas.

En la Figura 6, podemos darnos cuenta de los valores que sobresalen en estas 3 variables. Primero “daytime_sleep_minutes” tiene un máximo de 297 minutos lo cual es mucho para dormir durante el día, además de ser el único valor con esta magnitud, por lo cual se va a eliminar la instancia con ese valor.

Después en la variable “sleep_midpoint_minutes” podemos ver como también llega a haber outliers, pero en este caso, no vamos a eliminar ni cambiar nada ya que hay varios valores cerca y no lo vamos a catalogar como un error o algo extraño.

Finalmente, en el boxplot de “bedtime_variability” encontramos algunos outliers a partir de un valor de 5, al ser muy pocos valores, se van a considerar como outliers, por lo que se borrarán aquellas instancias con un valor de 5 o mayor en esta variable.

Con esto daríamos por terminado el proceso de Transformación y pasaríamos a corroborar que los datos están como queremos.

C. EDA Final

Se tomó la decisión de tener un EDA Inicial y uno Final con el objetivo de en un inicio observar cómo estaban los datos y en este caso, el final es para poder corroborar lo hecho en el ETL y poder decidir si pasar todas las variables a nuestro modelo.

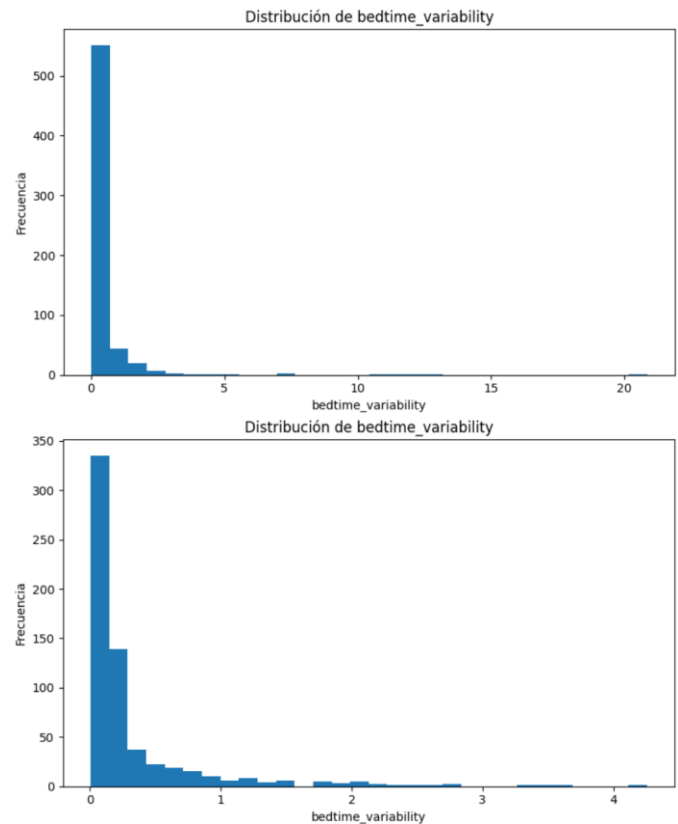


Figura 7. Comparación de Distribuciones

Se pueden apreciar 2 gráficas en la Figura 7, la de arriba pertenece al EDA Inicial, antes de hacer el ETL y la de abajo es ya después del ETL, en donde podemos notar que al borrar los outliers mejoró la distribución de los datos.

Este fue el primer paso para corroborar lo hecho y contrastarlo

con el inicio, también se volvió a obtener una matriz de correlación, pero es mejor verla desde los datos.

```
Correlación con term_gpa
term_gpa      1.000000
prior_gpa     0.640222
avg_sleep_hours 0.176082
nights_tracked_fraction 0.100847
daytime_sleep_minutes -0.099399
first_generation -0.138393
underrepresented -0.176749
bedtime_variability -0.214564
sleep_midpoint_minutes -0.219471
Name: term_gpa, dtype: float64
```

Figura 8. Datos de la matriz de correlación.

Con estos datos contrastados con la variable a predecir “term_gpa” podemos ver que hay algunas variables que no están muy relacionadas, esto no indica que las vamos a descartar automáticamente del modelo, pero es algo a tener en cuenta, más cuando se termine el primer modelo y se intente mejorar.

Finalmente obtuvimos correlación entre las variables independientes para buscar si hay alguna que se “repita” o sea redundante con otra, se obtuvo que no había una correlación mayor de 0.7 entre las variables, el cual fue nuestro colchón para activar o no la alarma.

III. MODELO SIN FRAMEWORK

Al enfrentarnos con un problema de regresión, el modelo a seguir en este proyecto tiene de base la Regresión Lineal Multivariable.

A. Preprocesamiento

Antes de pasar a la implementación del modelo, después de haber terminado con el EDA y ETL, extraemos el nuevo dataset ya listo para su uso en el modelo, es lo que diría si no fuera importante el preprocesamiento, ya que en el logramos hacer lo siguiente,

- Transformamos la variable categórica “gender” a 0’s y 1’s.
- Separamos el dataset en X y Y, siendo X la matriz de las variables independientes. Y siendo la variable para predecir “term_gpa”.
- Separamos las X en 60% Train, 20% Test y 20% Validation.
- Finalmente estandarizamos todos los valores para que no haya variables que dominen a otras o que el gradiente se mueva de manera desigual.

B. Parámetros e Hiperparámetros

Empezando por los parámetros, a los pesos se decidió

inicializarlos bajo el método HE, el cual nos arroja la fórmula:

```
W1 = np.random.randn(n_features, n_hidden) * np.sqrt(2.0 / n_features)
```

Figura 9. Inicialización con HE

Lo que nos dice esta fórmula es que iniciemos con un valor aleatorio para después multiplicarlo por la raíz cuadrada de 2 entre el número de features. Lo que esto nos permite es que, en el primer recorrido de la red, las neuronas no mueran al tener una activación de 0, así nos aseguramos de que al inicio todo funcione bien.

Para los Hiperparámetros se decidió tener una capa oculta con 16 neuronas, learning rate de 0.05 y 2000 epochs. Se decidieron los valores de manera arbitraria y al probarlo funcionó bien.

C. Función de Activación ReLU

Se tomó la decisión de tener una función de activación, primero para poder agregarle complejidad al modelo y no solamente quedarnos con la capa en la que se ingresan las X y directamente pasa a la salida, sino tener también una capa oculta para que el modelo pueda dejar de ser simplemente lineal.

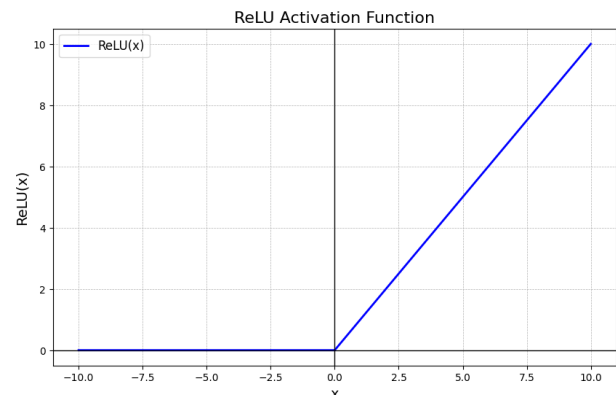


Figura 10. Función ReLU

Teniendo en cuenta lo anterior, se escogió ReLU como función de activación debido a su gran eficiencia computacional, además de que a lo largo de los años ha demostrado mejorar tareas en el aprendizaje profundo.

D. Regresión Lineal Multivariable

Llegamos al punto de explicar lo importante de este modelo, ya que, a pesar de escoger la función de costo, activación, por una razón, digamos que siguen siendo decisiones comunes y no se alejan a lo que realmente se hace con normalidad. En cambio, escoger una función de hipótesis sí que es algo más único, ya que cada problema es diferente. En esta ocasión, partiremos de una regresión lineal multivariable, ya que nos permite resolver problemas de regresión como es el caso, es simple computacionalmente hablando y multivariable porque

contamos con muchas X, las cuales nos permitirán predecir la Y con mayor precisión en lugar de enfocarnos solo en una X.

E. Función de Costo MSE

Después de que estuvimos recorriendo cada neurona y calculando predicciones con la ayuda de nuestra función de hipótesis, llegamos a la capa de salida en donde necesitamos calcular la función de pérdida o costo para poder saber que tan alejados estamos de un valor ideal.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2$$

Figura 11. Fórmula de MSE (Mean Square Error)

Para ello, nuestra función de costo será MSE (Mean Square Error), utilizamos MSE en lugar de MAE ya que nos permite distinguir entre valores positivos y negativos, cosa que MAE nos la complica más, ya que al arrojar un valor absoluto no sabremos que era antes del absoluto. La elección se cerró a estas 2 opciones porque ambas nos ayudan en problemas de regresión, a diferencia por ejemplo de Cross Entropy la cual es para problemas de clasificación más que para un valor continuo.

F. Back Propagation

Una vez que recorrimos toda la red al inicio, tenemos que dar vuelta atrás para poder saber en dónde tenemos que ajustar los pesos y los biases y así obtener la menor pérdida que podamos lograr, esto gracias a los epochs que son las vueltas que le damos a todo el dataset en este caso, ya que, al separar en Train, Test y Validation quedamos con un Train algo pequeño por lo que no hay mini-batches, así que se da una vuelta completa a todos los datos de entrada.

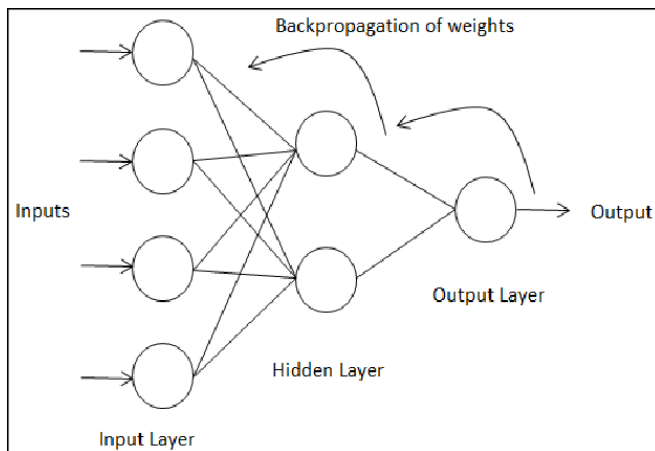


Figura 12. Back Propagation

En resumen, Back Propagation es un paso imposible de dejar de lado si se quiere llegar a un modelo que logre predecir con

cercanía a los valores reales. Back Propagation va de la mano con el siguiente punto del modelo.

G. Batch Gradient Descent

Gradient Descent no es otra cosa que una función de optimización que junto a back Propagation permite mejorar el modelo, por medio del cambio de los pesos sinápticos y los biases.

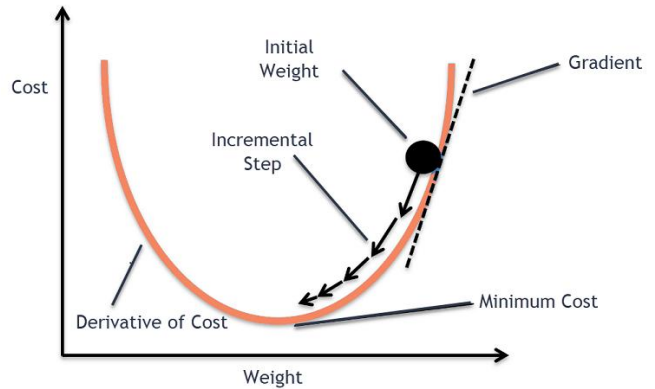


Figura 13. Gradient Descent

La forma en que logra optimizar nuestros parámetros es por medio del gradiente, es decir, utiliza las derivadas parciales de las neuronas y sus salidas Z y A hasta llegar al peso o bias que se va a ajustar.

Como se mencionó en el punto pasado, va de la mano con back Propagation para ir encontrando los gradientes desde el output hasta el inicio y así recorrer cada punto del modelo y calcular el cambio de los parámetros.

H. Resultados

Una vez explicado todas las decisiones que se tomaron para crear el modelo es que llegamos a los resultados, es decir, que descubrimos o a qué llegamos luego de aplicar todo lo mencionado en este artículo hasta este punto.

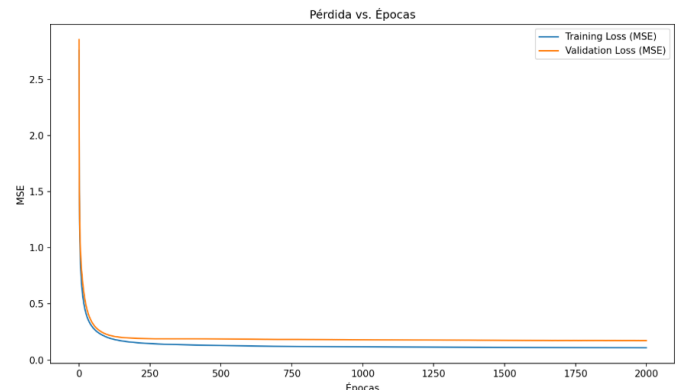


Figura 14. Train vs Validation

Primero observamos esta gráfica en busca de Overfitting, la cual parece no haber a pesar de que a partir del 250 epoch se

empiezan a separar, pero parece ser un valor mínimo, por ello, tenemos que irnos a los datos crudos, ya que visualmente parece ser poca la diferencia.

Train	MSE: 0.1093	RMSE: 0.3305	MAE: 0.2456	R ² : 0.5933
Validation	MSE: 0.1736	RMSE: 0.4167	MAE: 0.3117	R ² : 0.2038
Test	MSE: 0.1480	RMSE: 0.3847	MAE: 0.2808	R ² : 0.2193

Figura 15. Valores para medir el rendimiento del modelo

Por otra parte, en la consola imprimimos los valores que se muestran en la Figura 15 ya que nos permiten ver cómo se comporta nuestro modelo en realidad.

MSE y MAE nos muestran el error que tenemos al terminar con todos los epochs, si no tenemos alguna métrica con qué comparándolo es difícil comprenderlo. Por ello RMSE es un mejor valor para darnos una idea del funcionamiento, nos muestra el error medido en la misma unidad que el GPA, es decir, si un alumno tiene un GPA de 3.5, el modelo puede dar un valor de +/-0.38, un intervalo de (3.12, 3.88). Lo cual, al tener una escala de GPA de 0 a 4, puede ser un intervalo grande de error a pesar de ser decimal.

Finalmente, la R² nos dice que proporción de variabilidad del GPA explica el modelo. En la cual podemos notar cómo cambia drásticamente el valor de R² para Train que para Validation y Test, en Train se tiene 0.5933, mientras que en Test se tiene 0.21. Esto nos indica lo que no podíamos ver en la Figura 14, y es que notamos que existen un nivel medio de Overfitting, la red aprende bien, pero al momento de darle datos desconocidos con los que no fue entrenada, es que falla en predecir los valores reales.

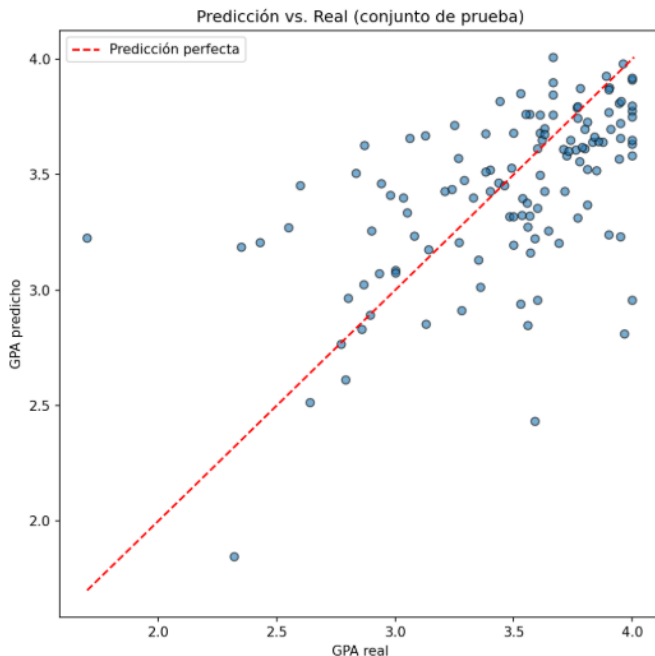


Figura 16. Predicción vs Real

La última gráfica u output que se generó en el código es la mostrada en la Figura 16, en ella podemos ver la comparación

de los valores reales en el eje x, contra los valores predichos por nuestro modelo, en ella podemos notar que hay muchos valores que están lejos de su valor real, otros valores que se acercan y otros pocos que, sí que llegan a tener el valor correcto, esto nos dice que hay una gran dispersión, muchos de los puntos están lejos de la línea roja que presenta el valor perfecto o deseado al predecir los reales.

En un inicio creí que se trataba de una especie de underfitting ya que pareciera que el modelo no es lo suficientemente complejo para poder predecir correctamente con los valores reales, pero justamente con el valor de R² de Train obtenido en la Figura 15, es que podemos descartar esta idea, ya que la mala predicción de los datos en test se debe a esta caída del R² en Test.

I. Observaciones

Continuando con lo visto en los resultados es necesario analizar lo que se logra ver.

- El modelo sobreestima a los estudiantes con GPA bajo (menor a 3.0), es decir, estudiantes con GPA real de 1.7 pudieron obtener un valor predicho de 3.2.
- Las predicciones caen en un rango de 2.8 a 4.0, cuando los valores reales están de 1.7 a 4.0, esto se complementa con el punto anterior.
- Pasa lo mismo con los estudiantes de alto rendimiento, ya que los sigue sobreestimando, va de rangos de 3.5 a 4.0 cuando en la realidad están en 2.4 a 4.0.
- Si se llegara a usar este modelo para poderle seguir el paso a un estudiante que está teniendo GPA's bajos, le estimaría un valor mucho más alto de lo que probablemente vaya a conseguir.

J. Bias y Varianza

Ya que encontramos que nuestro modelo cuenta con Overfitting, solo nos queda analizar su nivel de bias y varianza y así entender completamente cómo se está desempeñando.

Comenzando por el bias, estamos tomando en cuenta nuestro MSE de Train, en donde obtuvimos un valor de 0.1093, comparándolo con nuestra escala de valor a predecir "term_gpa" que va de 0.35 a 4.0, podría decirse que está cerca de ser un bias medio, pero al detectar Overfitting gracias al R², lo dejaremos como un bias bajo.

Por otro lado, la varianza la podemos estimar igualmente con MSE del Train, pero restándole al MSE de Validation, en donde porcentualmente obtenemos una diferencia del 59% aproximadamente, teniendo en cuenta esto es que lo catalogamos como una varianza media, justo como lo habíamos dicho anteriormente, en donde detectamos Overfitting, pero no un valor colosal, y lo comprobamos al

encontrar una varianza media y no alta.

Gracias a lo analizado en nuestros resultados es que podemos concluir que este modelo inicial sin el uso de algún framework presenta una varianza media, lo que implica Overfitting en él, para resolver esto es necesario aplicar alguna regularización al modelo y así bajar el Overfitting, pero antes de ello es necesario ver cómo se comporta con otro algoritmo, pero en este caso sí utilizando un framework.

IV. MODELO CON FRAMEWORK

Para poder abrir el panorama de este artículo es que se tomará en cuenta un segundo modelo implementando un algoritmo diferente al anterior y así encontrar el que se ajuste mejor a nuestros datos y así tener un modelo que haga mejores predicciones.

Para este segundo modelo se implementará Random Forest de una manera muy simple, ya que se quiere ver cómo es que se comporta el modelo así, para después crear una mejora de este modelo ya con límites como max Depth, etc.

A. Preprocesamiento

Igualmente, el primer paso siempre es el preprocesamiento en donde podremos ajustar los datos para que sean interpretables por el modelo.

Lo primero fue cargar nuestros datos igual a como los teníamos después del EDA y ETL, es decir, los cargamos igual a como los cargamos en el modelo anterior.

También transformamos nuestra variable categórica “gender” a 0’s y 1’s y dividimos nuestras X y Y.

Finalmente dividimos en 60% Train, 20% Validation y 20% Test, igual al anterior modelo. Un detalle importante es que con Random Forest no tuvimos que estandarizar nada ya que este trabaja por división de umbrales.

B. Parámetros e Hiperparámetros

Como se mencionó anteriormente, la intención de este modelo es probar un Random Forest de la manera más simple posible, por eso es por lo que se eligió un “bosque” con 100 árboles ya que es el valor por defecto de sklearn, de igual forma se inicializa una semilla aleatoria pero que se pueda replicar con las iteraciones que pueda haber después.

C. Explicación del modelo

El Random Forest parte de los árboles de decisión, pero en lugar de trabajar de manera independiente, colaboran unos con otros para que las predicciones que realizan sean más precisas.

Entonces, ¿qué son los árboles de decisión? Pues analizan diferentes variables y dividen los datos en ramas para clasificar o predecir resultados, pero cuando se trabaja con uno solo puede ser sensible a los cambios del conjunto de entrenamiento, por lo que habrá errores de generalización. Lo que hace Random Forest es combinar los resultados para reducir la varianza del modelo. Es por eso por lo que se eligió este método ya que detectamos una varianza Media en el de regresión lineal [7].

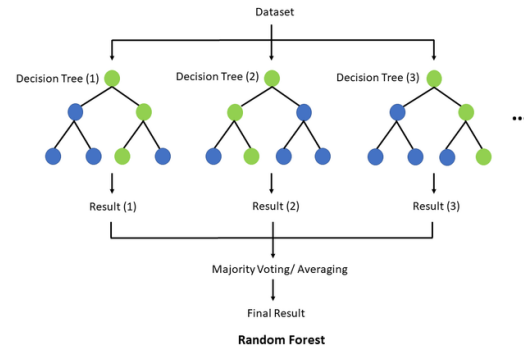


Figura 17. Random Forest

La forma en la que funciona Random Forest es la siguiente:

- El algoritmo selecciona aleatoriamente varias muestras del conjunto original de datos, a este proceso se le conoce como bootstrap sampling.
- Durante el entrenamiento, cada árbol elige aleatoriamente un subconjunto de características en cada división para reducir la correlación entre árboles.
- Cada muestra bootstrap genera un árbol de decisión independiente que se entrena sin poderse para que cada uno capte patrones específicos.
- Finalmente, el resultado se obtiene con el cálculo de la media de las predicciones de todos los árboles [7].

D. Resultados

Ahora teniendo en cuenta qué es lo que hace Random Forest, es momento de ver qué resultados obtuvimos con este método y si es que es mejor al método de regresión lineal.

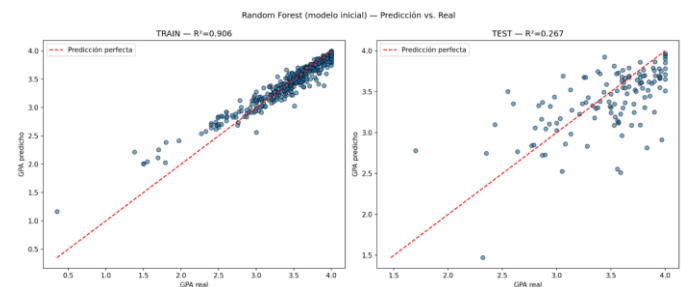


Figura 18. Predicción vs Real de Train y Test

Como se puede ver en la Figura 18, los resultados no son nada prometedores, lo que se ve es la comparativa de valores predichos contra los reales para Train y para Test, en ella

volvemos a encontrar el mismo problema que en el modelo anterior, ya que fácilmente se nota que tenemos un gran problema de Overfitting, mucho mayor al que tuvimos con la regresión lineal, es tan severo que solo con ver la gráfica lo sabemos, en el anterior dudábamos si se apreciaba o no Overfitting, en este vaya que no queda duda alguna.

La razón de que haya una diferencia tan abismal puede deberse a la falta de límites en este modelo como lo son max Depth y/o min samples leaf. Ya que como recordarán se decidió implementar un modelo de lo más sencillo posible para ver su comportamiento.

Train	MSE: 0.0254	RMSE: 0.1593	MAE: 0.1089	R ² : 0.9055
Validation	MSE: 0.1757	RMSE: 0.4191	MAE: 0.3184	R ² : 0.1945
Test	MSE: 0.1389	RMSE: 0.3727	MAE: 0.2795	R ² : 0.2674

Figura 19. Valores para medir el rendimiento del modelo

Con la figura 19 podemos visualizar de mejor forma que tan grave es nuestro Overfitting y efectivamente, es muy grave, Train cuenta con un R² de 0.9055, mientras que Test tiene un R² de 0.2674, es una gran caída. Esto lo que nos muestra es que el modelo aprende de una manera muy buena, llegando a ser casi irreal, pero cuando lo ponemos a predecir valores que estaban fuera del entrenamiento, no sabe como actuar y da valores muy malos comparado a lo que su desempeño había sido en el entrenamiento.

Pero no podemos dejar de lado los errores, si el R² es un buen indicador, pero también en los errores como en el RMSE podemos ver cómo se comporta la predicción en las 3 instancias del modelo, por ejemplo en Train observamos como es que en promedio varían las predicciones +/- 0.1593 puntos del GPA, lo cual es muy bueno, porque se muestra que varía poco, el problema llega cuando observamos ahora Test, en donde se tiene un RMSE de 0.3727, es una variabilidad de más del doble a cómo lo tiene Train, otro indicativo más del Overfitting severo del modelo.

E. Observaciones

Las observaciones que podemos destacar son las siguientes:

- El que no haya ningún límite de profundidad permite que cada uno de los 100 árboles crezca hasta que las hojas contengan muy pocas muestras.
- Que los resultados entre Test y Validation sean próximos es algo bueno ya que confirma que el Split es razonable.

F. Bias y Varianza

Para terminar de analizar este modelo hay que encontrar en que medida se presenta su varianza y bias, que ya nos podemos hacer una idea de cómo será debido al Overfitting.

El bias lo vamos a estar midiendo con el MSE del Train, donde hay un valor de 0.0254, si se recuerda lo visto en el modelo pasado, que teníamos 0.1 y se dudaba si entraba como

media o baja, ahora con un valor 0.02 no queda duda que estamos ante un bias bajo.

La varianza por otra parte es lo contrario a bajo, ya que, si comparamos el MSE de Train con el de Validation, tenemos una diferencia porcentual del 590% aproximadamente, esto claramente nos muestra que se cuenta con una varianza alta, simplemente ayudándonos a confirmar lo que ya se sabía, solo en caso de si aún quedaba duda.

Ya que pudimos analizar los resultados de este modelo, definitivamente nos dimos cuenta de que hay que hacer cambios en el, afectó mucho el hecho de que fuera tan simple y no agregásemos límites de profundidad. Por lo que hay que hacerle una mejora, pero antes de ello, con la R² de Test se ve que es mayor que la del modelo pasado, pero no es por mucho, lo que hace pensar que quizá haya que mejorar ambos a ver al final cuál de los 2 gana, no fue una victoria aplastante de Random Forest como para olvidarnos de la Regresión lineal, lo justo sería darles la oportunidad a ambos métodos y descubrir cuál termina con mejores predicciones.

V. MEJORA MODELO SIN FRAMEWORK

Antes de pasar de lleno con los cambios hechos en esta mejora del modelo sin Framework, es necesario explicar lo que es la regularización L2, ya que fue uno de los cambios aplicados en esta mejora.

Para entender L2 es necesario comprender lo que significa regularización en Machine Learning, la regularización lo que hace es “introducir una restricción en el modelo para desalentar un exceso de complejidad. Al hacerlo, se obliga al algoritmo a encontrar una solución más simple y, por lo tanto, más generalizable” [8]. Esto lo que ocasiona es que, en vez de enfocarse en patrones muy específicos, el modelo puede enfocarse en tendencias.

Teniendo esto en cuenta, lo que hace L2 es penalizar los coeficientes para poder mantener el modelo bajo control [8]. Permite que se distribuya de manera uniforme la magnitud de los coeficientes. Esto evita que se enfoque solamente en pocas características.

A. Cambios

Este modelo consistió en hacer cambios hasta encontrar la opción en donde los datos se predijeran de mejor manera, es por eso por lo que uno de los cambios hechos fue retomando la Figura 3, en la página 3 de este artículo.

En esta Figura podemos ver las variables que más relación tienen con el GPA, pero la mayoría de ellas las eliminamos por razones explicadas en el mismo ETL, finalmente nos quedamos con solamente 2 variables que muestran una correlación positiva con el GPA, esas son “prior_gpa” y “avg_sleep_hours”.

Correlación con GPA	
term_gpa	1.000000
prior_gpa	0.638025
gpa_change	0.548917
avg_sleep_minutes	0.201674
avg_sleep_hours	0.201542
student_id	0.196675
term_load_z	0.140652
nights_tracked_fraction	0.074180
term_units	-0.030672
bedtime_variability	-0.035967
study	-0.056451
first_generation	-0.135061
daytime_sleep_minutes	-0.153059
underrepresented	-0.170013
sleep_midpoint_minutes	-0.194537
under_6h_sleep	-0.199581
Name: term_gpa, dtype: float64	

Figura 3. Valores de Correlación con GPA.

Por lo que en el modelo solo nos quedamos con estos 2 features para ver cómo funcionaba el modelo y el resultado fue mejor a cuando le dejamos todas las variables, con ellas se obtuvo un R^2 de 0.36 mientras que con 2 variables fue de 0.41, ambas hablando de Test.

El siguiente cambio aplicado en la mejora del modelo fue la regularización L2, explicada anteriormente. Lo implementamos debido a que ayuda con el Overfitting el cual era nuestro problema en el modelo inicial, con un valor lambda de 20 ya que fue el que mejores resultados arrojó, pudiendo bajar el MSE de validación.

El último cambio fue la creación de un early stopping de los epochs para evitar calcular el mejor epoch a mano y mejor hacer que se detenga después de que no encuentre mejoras después de 800 epochs y así arroja el último mejor dato, no el último dato después de 800 epochs.

B. Resultados

Algunos de los resultados ya se mencionaron en la sección anterior, pero ahora es momento de profundizar en lo obtenido y ver la mejoría del modelo.

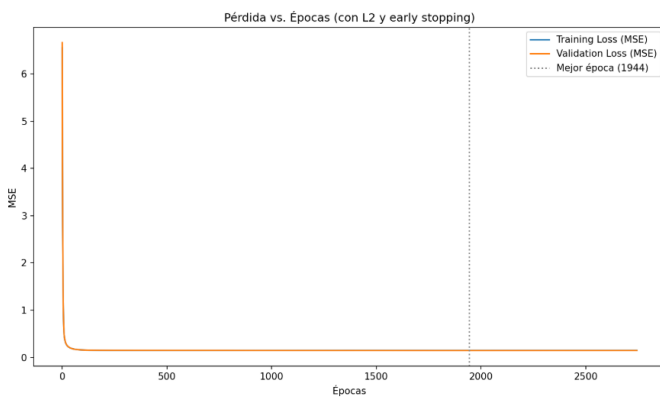


Figura 20. Train loss junto con Validation loss

Lo primero que podemos observar es cómo al comparar las pérdidas de Train y Validation, notamos que ahora si están juntas, no como en el modelo inicial que las veíamos algo

despegadas, en este caso no se observa ni una separación, por lo que visualmente podríamos descartar Overfitting, pero mejor para no errar hay que pasar a los datos reales y no visuales.

Train	MSE: 0.1475	RMSE: 0.3841	MAE: 0.2713	R^2 : 0.4508
Validation	MSE: 0.1436	RMSE: 0.3789	MAE: 0.2880	R^2 : 0.3417
Test	MSE: 0.1101	RMSE: 0.3318	MAE: 0.2529	R^2 : 0.4193

Figura 21. Valores para medir el rendimiento del modelo

Ahora que vemos los valores de R^2 podemos notar la diferencia que hay entre este modelo mejorado y el inicial, ahora la brecha entre R^2 de Train y de Test es mucho menor, siendo Train de 0.4508 y Test 0.4193, mucho mejor a cuando se tenía 0.5933 en Train y 0.2193 en Test, la brecha disminuyó y mejoró la predicción en el test. También mejoró Validation, aunque este se queda un poco debajo a comparación de los otros 2.

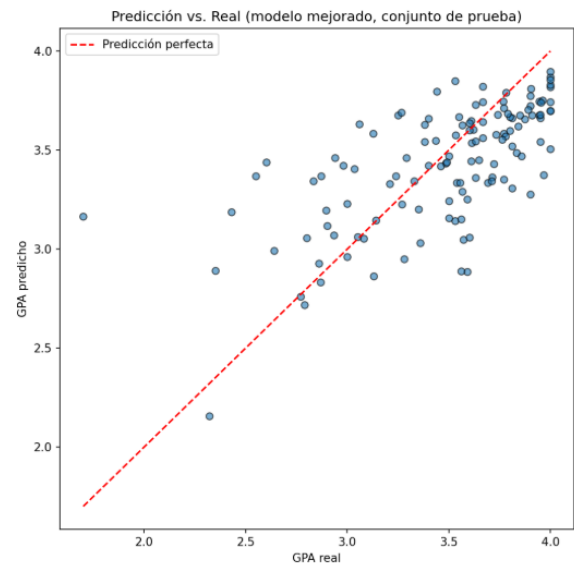


Figura 22. Predicción vs Real Mejorado

Finalmente, en la Figura 22 podemos ver cómo se comporta nuestra predicción con los valores reales, seguimos viendo que aún hay valores que no están cerca de la predicción real, mientras que hay otros que si logran estar en la línea de la predicción perfecta. Esto incidencia en nuestro R^2 ya que justamente nos dice que aproximadamente se está mostrando un 41% de la variabilidad de los datos, con esto podríamos estar ante una situación en la que nuestras variables no pueden predecir el GPA, no hay una relación tan directa, por lo que estamos llegando a un tope de nuestro modelo, en donde no depende tanto si es robusto o está sobreentrenado, etc. Sino que con los datos de entrada que hay, esto es lo que se puede lograr.

C. Bias y Varianza

Solamente hace falta revisar el bias y varianza de nuestro modelo para revisar qué tal quedó.

En el bias podemos notar que se encuentra en un rango medio, esto debido a que lo estamos estimando a partir del MSE de Train el cual tiene valor de 0.1475, que, si recordamos, lo evaluamos a partir de un rango de 0.35 a 4.0 que corresponde al GPA, por lo que en este caso con ese valor de MSE lo consideramos medio, pero sabemos que si le aumentamos la complejidad del modelo entraremos en Overfitting como estaba anteriormente. Anteriormente se probó que agregando de vuelta las variables que teníamos en el modelo inicial obtenemos un R^2 más baja, si subimos el número de neuronas también se volverá muy complejo para solo 2 variables de entrada. Así que esto podría estar confirmando lo dicho anteriormente, que se llegó a un tope debido a la naturaleza de los datos de entrada.

Por otro lado, la varianza, ha bajado considerablemente porque si le restamos MSE de Validation al MSE de Train, obtenemos una diferencia porcentual del -2% aproximadamente, lo que nos arroja una varianza baja, siendo que anteriormente había una diferencia entre estas 2 de 59%. Es una gran mejoría que nos indica que solucionamos el problema de Overfitting.

Antes de cerrar este artículo es fundamental también mejorar el modelo de Random Forest para poder encontrar el modelo más funcional de los 2.

VI. MEJORA MODELO CON FRAMEWORK

Algo importante de Random Forest que pasé por alto anteriormente debido a que no lo iba a utilizar en ese momento, son los Hiperparámetros del Random Forest, son distintos a los que utiliza la regresión lineal, por ello es importante entender a qué se refieren.

El primer Hiperparámetro es “n_estimators”, este se refiere al número de árboles que se utilizaran en el Random Forest.

Después tenemos “max_depth”, el cual indica la profundidad máxima que puede alcanzar cada árbol.

Finalmente tenemos “min_samples_leaf” el cual dice el número mínimo de muestras necesarias para estar en una hoja y digo finalmente no porque no haya más Hiperparámetros, sino porque estos serán los que se usarán, pero hay más [9].

A. Cambios

El primer cambio al igual que en el modelo mejorado sin framework, es dejar solamente 2 variables, siendo estas “prior_gpa” y “avg_sleep_hours”. Esto por exactamente la misma razón, se tienen mejores resultados cuando dejamos estas 2 variables, parece que dejar todas las que teníamos genera ruido en el modelo y baja su R^2 . Ya que en Test el de 2 variables arroja un valor de 0.39 y el que tiene todas las variables uno de 0.31 aproximadamente.

El segundo cambio forma parte de una serie de cambios en los hiperparámetros, lo que se hizo fue implementar una grid search en donde dentro de una serie de combinaciones se escoge la mejor opción, concretamente 48 combinaciones. Nos pudimos permitir utilizarlo debido a las pocas combinaciones que se cuentan, si tuviéramos más parámetros sería muy tardado y no tendría nada de bueno utilizar esta técnica. Se probó n_estimators [100,300,500], max_depth [2,3,5,8] y min_samples_leaf [5,10,20,30]. La forma de hacerlo es por medio de un for en el cual se va iterando cada sobre cada caso y se prueba el modelo con el R^2 de Validation y el que tenga mejor valor es la combinación ganadora, en este caso fue:

- n_estimators: 500
- max depth: 3
- min sample min: 10

Además, claro de tener nuestro generador de semilla aleatoria replicable. Esos fueron todos los cambios con los que se obtuvieron buenos resultados.

B. Resultados

Encontramos que logramos bajar el Overfitting extremo del modelo inicial de tal forma que las R^2 de Train y Test se asemejen en valor, acabando así con el Overfit.

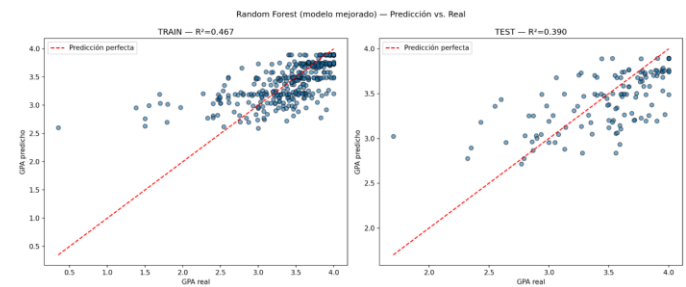


Figura 23. Predicción vs Real de Train y Test

En la Figura 23 podemos seguir viendo algo de diferencia en la predicción de Test y Train, aunque si recordamos la Figura 18 esta diferencia no es tanta como aquella, aunque es mejor ver un valor en número para demostrar completamente si es que sí se logró reducir la brecha entre ambas R^2 .

Train	MSE: 0.1431	RMSE: 0.3783	MAE: 0.2579	R^2 : 0.4674
Validation	MSE: 0.1442	RMSE: 0.3798	MAE: 0.2925	R^2 : 0.3386
Test	MSE: 0.1157	RMSE: 0.3401	MAE: 0.2605	R^2 : 0.3900

Figura 24. Valores para medir el rendimiento del modelo

En la Figura 24 vemos una gran mejoría con respecto al modelo inicial ya que teníamos un R^2 en Train de 0.955 y ahora bajó a 0.4674, con lo que podemos decir que si logramos eliminar el Overfitting presentado anteriormente, ya que además subieron las R^2 de Validation Y test, dejando a Test con un valor de 0.39, dejándolo ya más cerca de Train.

Además, podemos checar los valores de RMSE de los 3 y se puede notar como están en un rango cuando antes había una diferencia de más del doble.

C. Bias y Varianza

Parece ser que todo está yendo bien, pero la última prueba siempre es revisar qué tal está el bias y varianza del modelo y entender mejor su comportamiento.

Como de costumbre, para el bias tomamos el MSE de Train en donde tenemos 0.1431, lo cual lo tomaremos como una varianza media, de nueva cuenta se repite lo que sucedió en el modelo mejorado de regresión lineal, por lo que llegamos a la misma conclusión, si intentamos volver el modelo más complejo volveremos a donde estábamos, con Overfitting, por lo que se puede asumir que llegamos a un techo del dataset, en donde ya no se puede mejorar por la simple naturaleza de los datos.

Por otro lado, la varianza nos confirma que eliminamos el Overfitting de nuestro modelo, al tomar el MSE de Train con el de Validation y obtenemos la diferencia, obtenemos en porcentaje una diferencia de 0.80%, lo cual es muy bajo, y más teniendo en cuenta que antes se tenía una diferencia del 590%. Ahora con una varianza baja confirmada, también podemos confirmar que se elimina esa enorme diferencia de Train con las otras 2, borrando el Overfitting.

Terminamos de analizar este modelo y encontramos la misma situación que se presenta en el modelo de regresión lineal, ya que encontramos un bias medio, pero venimos de tener Overfit por lo que intentar arreglarlo nos volvería a colocar en esa posición, entonces nos quedaría la duda, si ambos tienen el mismo conflicto, ¿cuál es mejor modelo?

VII. DISCUSIÓN

Está claro que el modelo mejorado de Regresión tiene la ventaja al tener un R^2 de 0.41, mientras que Random Forest tiene uno de 0.39, contrastándolos así, nos podemos dar cuenta que son valores muy cercanos por lo que nos despierta la duda de si pudo haber sido suerte de la semilla aleatoria utilizada, por lo que se decidió probar con 8 semillas diferentes para descartar algún tipo de suerte y decantarnos por un modelo de una vez por todas.

	R^2 Promedio	Desviación Estándar
Regresión Lineal	0.388	0.024
Random Forest	0.359	0.029

Tabla 3. Regresión lineal o Random Forest

Después de correr 8 pruebas, el modelo con Regresión lineal obtuvo un R^2 superior a Random Forest en 7 de las 8 ocasiones, además de tener en promedio un R^2 mayor. Pero no solo fue eso, agregamos el factor de la desviación estándar para ver cuál variaba menos a lo largo de los splits y de nueva cuenta gana la Regresión Lineal.

Con estas pruebas es que se llega al punto de decantarse mejor por el modelo mejorado de regresión lineal ya que podemos

explicar mejor la variabilidad del GPA a diferencia del modelo Random Forest.

VIII. CONCLUSIÓN

Se aplicaron 2 modelos de Machine Learning y el R^2 más alta que se obtuvo después de incluso mejorar ambos modelos fue de 0.41, lo que significa que el mejor modelo solamente logra explicar el 41% de la variabilidad del GPA. Esto nos hace pensar si hay algo más allá del modelo ya que dentro de las mejoras se hicieron pruebas con distintos valores de parámetros y a lo mejor que se pudo llegar fue a eso, esto nos lleva a recordar lo que estamos analizando, se busca encontrar una relación entre las horas de sueño y el GPA de los estudiantes, lo cual si lo extrapolas a la realidad te das cuenta que puede ser que pase que si no duermes repruebas la materia, pero la realidad es que no es una relación lineal que suceda en la realidad, por lo que se puede llegar a concluir que se llega a este valor de R^2 debido a la naturaleza de los datos, ya que se tiene un techo debido a esto. Esto hace sentido con el bias medio que se obtuvo en el análisis de resultados, ya que un bias medio puede querernos decir que se puede estar presentando Underfitting, pero la realidad es que no es que haya Underfitting porque el modelo no logre hacer predicciones correctas, sino que se tiene un techo por cómo las variables del dataset pueden predecir el GPA.

Con lo cual, si no se puede predecir con certeza la calificación de un estudiante con respecto a sus horas de sueño, ¿Para que puede servir el modelo? La realidad es que no hay una respuesta concreta como tal, pero podemos decir que puede tener un buen uso si se sabe que tiene un margen de error. Es decir, si se es consciente de eso, puedes estimar tu GPA ingresando el anterior que tuviste y tus horas estimadas de sueño y así encontrar un posible rango en el que te encuentres ya dependerá de la persona encontrarle valor al rango o no, puede ayudarlo para que estudie más si es que no se puede permitir dormir mejor, es un mundo de posibilidades.

APÉNDICE

Repositorio del proyecto:

<https://github.com/AlejandroSH1/Implementaci-n-de-Aprendizaje-M-quina>

REFERENCIAS

- [1] Mucci, T. La historia de la IA. IBM. (2024, 21 octubre). <https://www.ibm.com/es-es/think/topics/history-of-artificial-intelligence>
- [2] S.A. Deep Blue. IBM. (s.f.). <https://www.ibm.com/history/deep-blue>
- [3] S.A. AlexNet — El primer gran éxito de una CNN que cambió el reconocimiento de imágenes. Zeromathai. (s.f.) <https://zeromathai.com/es/alexnet-es/>
- [4] S.A. ¿Qué es el Machine Learning? AWS Amazon. (s.f.). <https://aws.amazon.com/es/what-is/machine-learning/>
- [5] Mind Lab. Sleep vs. GPA in College. Kaggle. (s.f.). https://www.kaggle.com/datasets/kylefengkfeng209/sleep-vs-gpa-in-college?select=college_sleep_and_gpa.csv

- [6] Cuamba N, Flores G, González E. Depresión e Ideación Suicida en Estudiantes Universitarios: Un estudio Exploratorio con Perspectiva Género. UNAM, (2024). <https://ru.iiec.unam.mx/6634/>
- [7] S. A. Random Forest: qué es, cómo funciona y ejemplos de aplicación. INSEDI. (2025, 23 octubre). <https://www.inesdi.com/blog/random-forest-que-es/>
- [8] S. A. Regularización L2. Data Science Python Blog. (2024). <https://datasciencepythonblog.net/regularizacion-l2/>
- [9] García A. Optimización de Hiperparámetros de un clasificador de Machine Learning en Python. Panamá Hitek. (2023, 2 enero). <https://panamahitek.com/optimizacion-de-hiperparametros-machine-learning/>